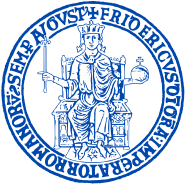


Documentazione Tecnica - WebTech's MemeMuseum



Corso: Tecnologie Web @ Università degli studi di Napoli Federico II (UNINA)

Traccia: B - WebTech's MemeMuseum

Studente: Oreste Leone N86/1980

1. Descrizione del Progetto

MemeMuseum è una Single Page Application (SPA) per la condivisione di meme. Supporta visualizzazione e ricerca di meme per utenti non autenticati e interazioni in aggiunta (upload, voti, commenti) per quelli autenticati.

L'architettura separa rigorosamente **Frontend** (React) e **Backend** (Express REST API), orchestrati via **Docker Compose** per garantire un ambiente riproducibile.

2. Tecnologie Utilizzate

Backend (/backend)

- **Node.js (v24 LTS) & Express.js:** Runtime e framework per API REST veloci e scalabili.
- **SQLite:** Database relazionale su file (zero-config) persistito via Docker Volume.
- **Prisma ORM:** Accesso al database type-safe con gestione automatica delle migrazioni.
- **Multer:** Middleware per la gestione dell'upload di immagini (salvate localmente).
- **JWT (JSON Web Tokens) & bcryptjs:** Autenticazione stateless sicura e hashing delle password.

Frontend (/frontend)

- **React (v18) + Vite:** Libreria UI a componenti e build tool di nuova generazione (HMR istantaneo).
- **TypeScript:** Linguaggio tipizzato utilizzato nell'intero stack per robustezza e manutenibilità.
- **Tailwind CSS + DaisyUI:** Utility-first CSS framework con componenti UI pre-stilizzati.
- **React Query (TanStack Query):** Gestione ottimizzata dello stato e recupero dati asincroni su cui basare i componenti React.
- **Context API:** Gestione dello stato globale client (es. sessione utente/Auth) per evitare "prop drilling".
- **Axios:** Client HTTP per le chiamate API REST.
- **React Router:** Gestione del routing client-side.

Infrastruttura & Testing

- **Docker & Docker Compose:** Containerizzazione dei servizi (Frontend, Backend) e gestione volumi.
- **Nginx:** Web server leggero per servire i file statici del frontend.
- **Cypress:** Framework per test End-to-End automatizzati.

3. Organizzazione del Codice e Pattern

Il progetto adotta diverse best practices per garantire pulizia e manutenibilità:

- **Shared Types:** Le interfacce TypeScript (DTO) e le costanti (es. `SortOrder`) sono condivise in `/src/shared` sia nel frontend che nel backend, garantendo coerenza nei contratti dati.
- **Backend Utilities:**
 - `asyncHandler`: Un wrapper per i controller che gestisce automaticamente le eccezioni asincrone, eliminando la necessità di blocchi `try-catch` ripetitivi.
 - Separazione netta tra Controller (gestione HTTP) e Service (logica di business).
- **Frontend Architecture:**
 - **Axios Interceptor:** Un'istanza centralizzata di Axios inietta automaticamente il token JWT in tutte le richieste, semplificando le chiamate API.
 - **Custom Hooks:** Logiche complesse (es. la gestione del voto in `useVote`) sono estratte dai componenti UI per migliorare la leggibilità.